

CSE422: Artificial intelligence

Assignment 02 [Spring 2026]

Name:		ID:		Section:	
-------	--	-----	--	----------	--

1. You are a delivery driver who must visit **8 cities exactly once** and return to the starting point. Each pair of cities has a known **distance** between them, as given in the following table. Your goal is to find the **shortest possible route** that visits all cities once and returns to the start. You will use a genetic algorithm to solve this problem.

	A	B	C	D	E	F	G	H
A	-	4	8	6	7	3	5	9
B	4	-	2	7	3	6	8	5
C	8	2	-	9	4	7	6	3
D	6	7	9	-	2	4	5	7
E	7	3	4	2	-	6	3	5
F	3	6	7	4	6	-	2	8
G	5	8	6	5	3	2	-	4
H	9	5	3	7	5	8	4	-

- Propose an encoding scheme to represent candidate solutions of the problem.
- Generate an initial population of 6 using the proposed encoding scheme.
- Propose a fitness function suitable for the problem.
- Run one iteration of Genetic Algorithm for this problem. Choose random crossover and mutation indices.
- Show the best solution after running one iteration.
- Show what the population for the next iteration will be using 2-elitism.

2. You are given a 4th order polynomial function: $f(x) = x^4 - 14x^2 + 24x + 30$. This function has many local maxima and minima, but our target is to find the global minimum value of the function. We will use a genetic algorithm for this.

- Propose an encoding scheme to represent candidate solutions of the problem. [Hint: your encoding must be such that it is possible to perform operations like cross-over and mutation. You could say $x_1 = 5, x_2 = 6$, ... is a valid encoding scheme, but how would we perform cross-over or mutation? So, as you can see, it is not actually a good encoding.]
- Generate an initial population of 4 using the proposed encoding scheme.
- Propose a fitness function suitable for the problem.
- Run one iteration of Genetic Algorithm for this problem. Choose random crossover and mutation indices.
- Show the best solution after running one iteration.
- Show what the population for the next iteration will be using 2-elitism.

3. You are tasked with designing a seating arrangement for a wedding reception with 8 guests. The guests have specific preferences about whom they would like to sit next to and whom they want to avoid. You need to create a seating arrangement for these guests around a circular table. Depending on the guests' preferred seating arrangements, the aim is to optimize their level of satisfaction. Consider the following constraints:

- Guest 1 (G1) prefers to sit next to Guest 2 and Guest 3 but wants to avoid Guest 4.
- Guest 2 (G2) prefers to sit next to Guest 1 and Guest 4 but wants to avoid Guest 5.
- Guest 3 (G3) prefers to sit next to Guest 1 and Guest 6 but wants to avoid Guest 7.
- Guest 4 (G4) prefers to sit next to Guest 2 and Guest 5 but wants to avoid Guest 8.
- Guest 5 (G5) prefers to sit next to Guest 4 and Guest 6 but wants to avoid Guest 1.
- Guest 6 (G6) prefers to sit next to Guest 3 and Guest 5 but wants to avoid Guest 2.
- Guest 7 (G7) prefers to sit next to Guest 8 but wants to avoid Guest 3.
- Guest 8 (G8) prefers to sit next to Guest 7 but wants to avoid Guest 4.

You have to solve the problem using a genetic algorithm.

- Encode the problem and create four parent chromosomes.
- Suggest a possible fitness function and choose the two fittest Chromosomes.
- Do you think the regular crossover (as in the n-queen problem) will work in this scenario? Explain why. [Hint: this is a permutation problem, just like the TSP one we saw in class. Each guest should only appear once in a permutation, but this constraint may be violated if we use regular crossover. What would be an appropriate crossover for these cases?]
- Show how to perform mutation in this context.

4. You are the surveillance officer of a museum. You want to bring the museum under CC-TV surveillance. You have THREE cameras, but you need to cover 20 distinct areas. Each camera has a CAPACITY, i.e. how many places (both towards its left and right) can a camera cover. For example, a camera with a CAPACITY of 3 can cover 3 places to its left, 3 places to its right, and the spot where it is placed (so altogether 7 places). The first two cameras have a CAPACITY of 2, and the remaining camera has a CAPACITY of 3. Based on this, you need to place the cameras at strategic positions so you can cover as many areas as possible. You will use a genetic algorithm for this.

- Design an encoding scheme for the problem.
- Design a suitable fitness function.
- Using your encoding scheme, show FOUR parents and calculate their fitness.
- Will single point crossover work for this problem? Explain.

5. A company uses a machine learning algorithm to recruit its employees. However, there are some complaints that the algorithm has a bias based on certain demographic areas, gender, race etc, meaning that two candidates with similar qualities may not be given the same opportunity. To identify if any bias exists the company decides to divide a group of candidates into two subsets such that candidates with similar qualifications go to distinct subsets so they can find out if candidates with similar qualifications are getting different outcomes. The similarity score between 6 candidates (A1... A6) is calculated based on skills, education, experience and is given below:

According to the problem description for Question 5, you are trying to create a partition with specific traits:

The Penalty: The prompt states that if highly similar candidates are placed in the same subset, "the fitness should decrease."

The Reward: The natural opposite of this is that placing highly similar candidates in different subsets should make the fitness increase.

3. Building the Equation
To make the algorithm seek out the best arrangement, we use a simple Reward minus Penalty structure:

Formula: $Fitness = \Sigma(\text{Inter-subset similarities}) - \Sigma(\text{Intra-subset similarities})$

$\Sigma(\text{Inter-subset similarities})$ [The Reward]: We look at every pair of candidates where one is in Subset 1 and the other is in Subset 2. We add their similarity scores together. Because this is a positive number, the GA will try to maximize it by pushing candidates with high similarity scores into opposite groups.

$-\Sigma(\text{Intra-subset similarities})$ [The Penalty]: We look at every pair of candidates sharing the same subset. We add their similarity scores together, and then we subtract that total from the score.

Why this works perfectly:
Imagine a terrible candidate solution where the algorithm accidentally puts A1, A2, and A3 into Subset 1, and their similarity scores with each other are extremely high (e.g., 8, 9, and 7). Because they are in the same subset, that total of 24 is treated as a penalty. The fitness function will subtract 24 from the overall score, resulting in a very low (or even negative) fitness.

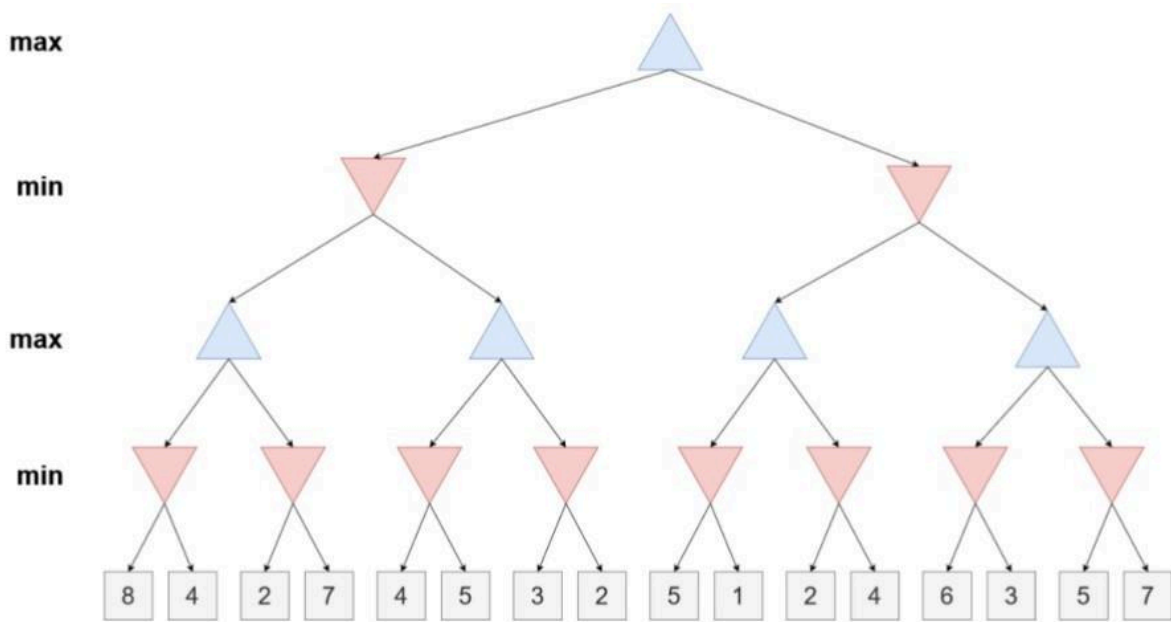
	A1	A2	A3	A4	A5	A6
A1	0	8	3	6	5	7
A2	8	0	7	4	2	3
A3	3	7	0	9	6	2
A4	6	4	9	0	7	1
A5	5	2	6	7	0	5
A6	7	3	2	1	5	0

The task is to partition the candidates into two subsets such that the similarity scores between candidates in different subsets is maximized and the similarity score between candidates in the same subset is minimized. For example, if two candidates are in two different subsets the fitness should increase, if they are in the same subset then the fitness should decrease.

- Propose an encoding scheme to represent the solution in a genetic algorithm. Explain using an example.
- Define a fitness function to evaluate how good a partition is. Use the example from (a) to explain.
- Demonstrate the crossover operation between two randomly generated parents.

6. Apply the Minimax algorithm with **alpha-beta** pruning on the following game tree to determine the best possible outcome for the MAX player at root node, assuming both MAX and MIN play optimally.

- Show alpha and beta values for each node.
- Show which branches can be pruned.



7. Suppose you are to design an **agent (circle)** that knows how to play a game of tic tac toe against the **user (cross)**. The following figure shows the current state of the game. Now the **agent has to decide which move to make** by using the minimax algorithm with alpha beta pruning.

O	O	X
	X	
O	X	

Determine which move would the agent prefer from the given state of the game. The final utilities will be $[(100 - \text{Depth}) * 1]$ for a win, $[(100 - \text{Depth}) * -1]$ for a loss and 0 for a draw. You must showcase the values of the nodes, α and β .

[Hint: Assuming the depth of the root node is 1, if a leaf node is found at the next level (depth 2) where the agent wins, then the utility for that leaf node from the agent's perspective would be $(100 - 2) * 1 = 98$. For a losing state it would have been $(100 - 2) * -1 = -98$]

The Core Rule of Minimax

MAX (The Agent/AI): This is the player whose point of view we are calculating. MAX wants to maximize the utility score to win the game.

MIN (The Opponent): This is the other player. We assume they are playing perfectly, meaning their goal is to minimize MAX's utility score.

Since we are figuring out the best move for the agent ('O'), the agent is MAX.

Since the user ('X') is the opponent trying to beat the agent, the user is MIN.

Depth 1 (Root Node): The problem asks which move the agent should make right now. This means it is currently the agent's ('O') turn. Therefore, the Root is a MAX node.

Depth 2: After 'O' makes a move, it becomes the user's ('X') turn. Therefore, all nodes at Depth 2 are evaluated by MIN.

Depth 3: After 'X' makes a move, it becomes 'O's turn again. Therefore, all nodes at Depth 3 are evaluated by MAX.